BIT MANIPULATION

→ <u>Bit representation</u> :-

- In C++, int – 32 bit

  int $x = 43 \longrightarrow$ 00000000000000000000101011

- Bit = $\cancel{b_1 b_2 b_3} \; b_k \cdots b_2 b_1 b_0$

  Number = $b_k 2^k + \cdots + b_2 2^2 + b_1 2^1 + b_0 2^0$

- Bit representation is either signed or unsigned.
- Usually a signed representation is used, which means that both +ve and -ve numbers can be represented.
- A signed variable of n bits – Numbers b/w -2^{n-1} to $2^{n-1}-1$

  Eg- int in C++ $\longrightarrow -2^{31}$ to $2^{31}-1$

- First bit in a signed representation is the sign of the number (0 for nonnegative and 1 for negative numbers) and the remaining n-1 bits contain the magnitude of the number.

- <u>Two's complement</u> :- To represent -ve numbers.
  - ↳ Calculate the magnitude in bits.
  - ↳ Invert all the bits
  - ↳ Add 1.

- $0 + 1 = 1$

  $1 + 1 = 10$

  $$\begin{array}{r} 011 \\ +\,1 \\ \hline 100 \end{array}$$

- In an unsigned representation, only non-negative numbers can be used, but the upper bound for the values is larger.

- An unsigned variable of n bits – Numbers – 0 and $2^n - 1$.
- In C++, unsigned int $\longrightarrow 0$ to $2^{32}-1$
- [Signed no. $-x$ = Unsigned no. $2^n - x$].

Scanned with OKEN Scanner

eg- int $x = -43$
unsigned int $y = x$;
cout << x << "\n"; // -43
cout << y << "\n"; // 4294967253 $\longrightarrow 2^{32} - 43$

- If a no. is larger than the upper bound of the bit representation, the number will overflow.

→ signed representation → Next no after $2^{n-1}-1$ is -2^{n-1}.

→ Unsigned representation → Next no. after $2^n - 1$ is 0.

eg- int $x = 2147483647 = 2^{31}-1$
cout << x << "\n"; // $2147483647 = 2^{31}-1$
x++;
cout << x << "\n"; // $-2147483648 = -2^{31}$

Bit Operations :-

① And operation (&) :- (Bit wise &) :-

0 & 1 = 0	all true → true
1 & 1 = 1	1 false → false
0 & 0 = 0	

- If a number is even, $\boxed{x \,\&\, 1 = 0}$ → x is even
- If a number is odd, $\boxed{x \,\&\, 1 = 1}$ → x is odd

} if(x & 1) 1 = true, 0 = false

⇒ x is divisible by 2^k exactly, when $\boxed{x \,\&\, (2^k - 1) = 0}$

What does it mean :-

→ A power of two (2^k) has exactly 1 bit.

→ Subtracting 1 flips that bit and all bits to the right.

	2^k	2^{k-1} (binary)
$2^3 =$	1000	0111
$2^5 =$	100000	011111

$\boxed{2^k - 1 = k \text{ ones in binary}}$

$x - 1$ means → "Turns the rightmost 1 bit into 0.
"Turns all bits to the right of it to 1."
"Leaves bits to the left unchanged."

Scanned with OKEN Scanner

→ ~~2^k means~~ Divisible by 2^k means the last k bits of x are all 0

eg. for k=3 (divisible by 8)

$x = 40 = 101000$ ✓ (last 3 bits are 0)

$x = 44 = 101100$ ✗ (lasts 3 bits are not all 0)

∴ $x \,\&\, (2^k - 1)$ checks whether last k bits of x are zero or not.

$x \,\&\, (\underbrace{2^k - 1}_{k \text{ ones bits}})$ → If last k bits of x are zero then $x \,\&\, (2^k - 1) = 0$, which means x is divisible by 2^k exactly.

Otherwise false.

② OR operation (Bitwise |) :-

1	|	1	= 1
0	|	1	= 1
0	|	0	= 0

1 true → true
all false → false

③ XOR operation (Bitwise ^) :-

1 where odd 1's are present ... else 0.

no. of 1s odd → 1
no. of 1s even → 0

eg.

$$\begin{array}{rl} & 10110 \quad (22) \\ \wedge & 11010 \quad (26) \\ \hline = & 01100 \quad (12) \end{array}$$

0	^	0	= 0
1	^	1	= 0
0	^	1	= 1

$a \wedge 0 = a$
$a \wedge a = 0$

④ NOT operation (Bitwise ~) :- All bits of x are inverted.

$\boxed{\sim x = -x - 1}$ → eg- $\sim 29 = -29 - 1 = -30$.

⑤ Bit shifts (Left shift << and Right shift >>) :-

- Left bit shift $x << k$ appends k zero bits to the number.
- Right bit shift $x >> k$ removes the last k bits from the number.

- $x << k$ → Multiplying by 2^k → eg- $1 << 2 = 100$
- $x >> k$ → Dividing by 2^k, then rounding down to an e... → eg- $100 >> 1 = 010$

Scanned with OKEN Scanner

Applications :-

(1.) check whether the number x has 1 or zero at the kth position.

- If $\boxed{x \,\&\, (1<<k) = 1}$ $\longrightarrow$ 1
- If $x \,\&\, (1<<k) = 0$ $\longrightarrow$ 0

$\boxed{\text{or}}$

- If $\boxed{(x>>k) \,\&\, 1 = 1}$ $\longrightarrow$ 1
- If $(x>>k) \,\&\, 1 = 0$ $\longrightarrow$ 0

(2.) Set k^{th} position to one :- $\boxed{x \mid (1<<k)}$

(3.) set k^{th} position to zero :- (Clearing the bit) $\boxed{x \,\&\, \sim(1<<k)}$

(4.) Invert the kth bit :- $\boxed{x \wedge (1<<k)}$
(Toggle the bit)

(5.) Set the last one bit (rightmost) of x to zero :- $\boxed{x \,\&\, (x-1)}$

$x = 12$

eg- $\rightarrow x = 1100$, $x-1 = 1011$

$x \,\&\, (x-1) = (1100) \,\&\, (1011) = 1000$

last bit set to zero

(6.) Invert all the bits after the last one bit :- $\boxed{x \mid (x-1)}$

Note:- A +ve no. x is a power of two exactly when $x \,\&\, (x-1) = 0$

Additional function:- g++ compiler provides function for counting bit

- __builtin_clz(x) : the number of zeros at the beginning of the number. (int)
- __builting_ctz(x) : number of zeroes at the end of the no.
- __builtin_popcount(x) : no. of ones in the number.
- ~~__builtin_popcoun~~
  __builtin_parity(x) : the parity (even or odd) of the number of ones.

- . . . . . use suffix ll .

Scanned with OKEN Scanner

Bitwise Tricks :-

(1.) Swap two numbers :-

Using XOR →

$a = a \wedge b$

$b = a \wedge b = (a \wedge b) \wedge b = a \wedge (b \wedge b) = a \wedge 0 = a$

$a = a \wedge b = (a \wedge b) \wedge b = (a \wedge b) \wedge a$ ~~= 0 ∧~~

$= (a \wedge a) \wedge b$

$= 0 \wedge b = b$

(2.) Count the number of set bits :-

```
count = 0
while (N != 0)
{
    N = N & (N-1)
    count++
}
return count
```

→ Set the last bit (rightmost) to zero.

TC → O(31)

But in most cases → < O(31)

(3) Minimum bitflips to convert a number :-

start = 10 → 1010

goal = 7 → 0111

$10 \wedge 7 = 1101$

Min 3 bit flips required

[ans = start ∧ goal]

```
count = 0
for ( i = 0 → 31)
{ if (ans & (1 << i))
      count++;
}
cout << count << endl;
```

TC = O(31)

⇒ or : count = __builtin_popcount(ans)

(4.) Using bits for long long :-

for checking 63 0 to (63-1) bits

```
for(int i=0; i<63; i++)
  if((1ll << i) & x)
```

→ we write ll or LL here.

Scanned with OKEN Scanner

(5) Using base n instead of 2 in binary system :-

Sometimes, we can represent binary in base of a no. 'n' insted of 2.

Then if $b_k b_{k-1} \cdots b_2 b_1 b_0$ is the binary (0 or 1)

Its decimal representation is

$$\left[b_k \cdot n^k + b_{k-1} \cdot n^{k-1} + \cdots + b_2 \cdot n^2 + b_1 \cdot n^1 + b_0 \cdot n^0 \right]$$

(6) Power set (How to create all subsets) :-

Eg- arr = {1, 2, 3}

subsets = { {}, {1}, {2}, {3}, {1, 2}, {2, 3}, {1, 3} }

→ [1 2 3] → n = arr.length() or no. of elements.

Bits → 2 1 0

Total no. of subsets = 2^n

nums & (1<<i)

0 →	0	0	0	→	[]
1 →	0	0	1	→	[1]
2 →	0	1	0	→	[2]
3 →	0	1	1	→	[1, 2]
4 →	1	0	0	→	[3]
5 →	1	0	1	→	[1, 3]
6 →	1	1	0	→	[2, 3]
7 →	1	1	1	→	[1, 2, 3]

Code:-

```
total = 1<<n        // 2^n   (n = no. of elements of arr)
subset = []
for (i=0 → total-1) → 2^n times
{   list = []
    for (i=0 → n-1)
    {       if (nums & (1<<i))
                list.add(nums[i])
    }
    subset.add(list)
}
```

TC → O(

(7) Find single number in an array having duplicate numbers and only one single number (Single Number - I) :-

$-10^5 < nums[i] < 10^5$

eg- nums = [4, 1, 2, 1, 2]

↳ n = no. of elements in array

>> Brute force :-

```
~~map<~~ ~~unordered~~ map <int, int> mpp
for (i=0 → n-1)
{
    mpp[nums[i]]++ ; → log m
}
for (auto it : mpp)
{
    if (it.second==1)
        return it.first;
}
```

N x log m

m

~~unordered map TC → O(1) avg, worst~~
→ O(n) worst

[4], 1	
[1], 2	
[2], 2	

m

map → TC → O(log m)

$m = \left(\frac{n}{2} + 1\right)$

= No. of unique elements

TC → O(N x log m + m)

SC = O(m)

>> Efficient :-

XOR → $a \wedge a = 0$

$a \wedge 0 = a$

$\therefore 4 \wedge 1 \wedge 2 \wedge 1 \wedge 2 = 4 \wedge (1 \wedge 1) \wedge (2 \wedge 2)$

$= 4 \wedge 0 \wedge 0$

$= 4$

Code :-

```
XOR = 0
for (i = 0 → n-1)
{
    XOR = XOR ^ nums[i];
}
return XOR;
```

→ TC → O(n)

Scanned with OKEN Scanner